

COMP/EECE 7/8745 Neural Networks

Assignment # 3

Spring 2026

Due: April 10, 2026

Total points: 100

Task 1 (Points: 50): The object of this project is to develop a simple Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU) models to predict Google stock prices. For this project, the datasets have been uploaded to CANVAS. The file names are as follows:

1. Google_Stock_Price_Train.csv
2. Google_Stock_Price_Test.csv

The reference codes have been uploaded to the CANVAS (Lecture 11).

Task 2 (Points: 50) Design and evaluate Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), and Transformer models for hotel description generation tasks.

Please consider the following steps to complete the tasks:

Step 1. Import the necessary libraries : PyTorch, keras, numpy, pandas, etc.

Step 2. Load the datasets ('Seattle_Hotels_address_description.csv') as panda dataframe.

Step 3. Import TOKENIZER from keras as follows:

- a.

```
from keras.preprocessing.text import Tokenizer
corpus = [x for x in all_descriptions] # all_description is the input's
description as list.
corpus[:1]
t = Tokenizer(num_words=None, filters='!"#$%&()*+,-
./:;<=>?@[\\]^_`{|}~\t\n', lower=True, split=' ', char_level=False,
oov_token=None, document_count=0)
t.fit_on_texts(corpus)
```

Step 4. Display all necessary logs from TOKENIZER object (t) (If needed)

```
# A dictionary of words and their counts.
print(t.word_counts)
# A dictionary of words and how many documents each appeared in.
print(t.word_docs)
# An integer count of the total number of documents that were used to fit
the Tokenizer (i.e. total number of documents)
print(t.document_count)
# A dictionary of words and their uniquely assigned integers.
```

```
print(t.word_index)
```

```
print('Found %s unique tokens.' % len(t.word_index))
```

Step 5. Once tokenizer is created then apply it on the corpus generated in **Step 3**.

```
# Tokenization
```

```
t = Tokenizer(num_words=None, filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n', lower=True, split=' ', char_level=False, oov_token=None, document_count=0)
```

```
def get_sequence_of_tokens(corpus):
```

```
    t.fit_on_texts(corpus)
```

```
    total_words = len(t.word_index) + 1
```

```
    input_sequences = []
```

```
    for line in corpus:
```

```
        token_list = t.texts_to_sequences([line])[0]
```

```
        for i in range(1, len(token_list)):
```

```
            n_gram_sequence = token_list[i+1]
```

```
            input_sequences.append(n_gram_sequence)
```

```
    return input_sequences, total_words
```

```
input_sequences, total_words = get_sequence_of_tokens(corpus)
```

```
print(input_sequences[:10])
```

```
print(total_words)
```

Step 6. Apply padding for making the consistent inputs size of “input_sequences” from **Step 5** using following function

```
# pad sequences
```

```
def generate_padded_sequences(input_sequences):
```

```
    max_sequence_len = max([len(x) for x in input_sequences])
```

```
    input_sequences = np.array(pad_sequences(input_sequences, maxlen = max_sequence_len, padding = 'pre'))
```

```
    predictors, label = input_sequences[:, :-1], input_sequences[:, -1]
```

```
    label = ku.to_categorical(label, num_classes = total_words)
```

```
    return predictors, label, max_sequence_len
```

```
predictors, label, max_sequence_len =
```

```
generate_padded_sequences(input_sequences)
```

Step 7. Design LSTM, GRU and Transformer models (Lecture_14.ipynb). The architecture of the LSTM and GRU models are as follows:

- i. Embedding (input_dim= total_words ,output_dim=10,
input_length=max_sequence_len – 1)→
SimpleRNN(100)→Dropout(0.1)→Dense(total_words,activation="softmax")
- ii. Embedding (input_dim= total_words ,output_dim=10,
input_length=max_sequence_len – 1)→
LSTM(100)→Dropout(0.1)→Dense(total_words,activation="softmax")
- iii. Embedding (input_dim= total_words ,output_dim=10,
input_length=max_sequence_len – 1)→
GRU(100)→Dropout(0.1)→Dense(total_words,activation="softmax")

Training criterions:

1. All models will be compiled with “categorical_crossentropy” loss and ADAMw optimization methods.
2. Train the models for 50 epochs.

Testing criterions:

1. Once the training is done, use the following function to create the descriptions:

```
def generate_text(seed_text, next_words, model, max_seq_len):  
    for _ in range(next_words):  
        token_list = t.texts_to_sequences([seed_text])[0]  
        token_list = pad_sequences([token_list], maxlen=max_seq_len-1,  
padding='pre')  
        predicted = np.argmax(model.predict(token_list), axis=-1)  
        output_word = "  
        for word,index in t.word_index.items():  
            if index == predicted:  
                output_word = word  
                break  
        seed_text = seed_text + " " + output_word  
    return seed_text.title()
```

2. For generating the hotel's description, you have to provide the “seed_text”, next_words → number of next words, model → trained model, max_sequence_len.

Example seed texts:

1. "hilton seattle downtown"
2. "best western seattle airport hotel"
3. "located in the heart of downtown seattle"

Generate the text outputs for all three models and compare the results in the reports.

Datasets in CANVAS:

- Task 1: ~/CANVAS/Datasets/Assignment_3/ Google_Stock_Price_Train.csv
- Task 1: ~/CANVAS/ Datasets/Assignment_3/ Google_Stock_Price_Test.csv
- Task 2 : ~/CANVAS/ Datasets/Assignment_3/ Seattle_Hotels_address_description.csv

Ref. codes available in CANVAS:

- Task 1: ~/CANVAS/Files/Ref.codes/Lecture_11.ipynb
- Task 2: ~/CANVAS/Files/Ref.codes/Lecture_12.ipynb
- Task 2: ~/CANVAS/Files/Ref.codes/Lecture_13.ipynb
- Task 2: ~/CANVAS/Files/Ref.codes/Lecture_14.ipynb

Final Note: You must submit **fully executable source code** for **both tasks** as part of this assignment.

Report Outline

1. Title Page

- Course Title
- Course Number
- Student Name
- Student ID
- Assignment Number
- Submission Date (Month, Year)

2. Abstract

(Starts on the second page)

- Brief problem statement
- Dataset and methodology overview
- Key results and performance highlights
- Main conclusions

3. Introduction

3.1 Problem Definition

- Description of the classification/regression task
- Motivation and real-world relevance

3.2 Background and Related Work

- Overview of CNN/DNN-based approaches
- Summary of relevant prior studies and models

3.3 Objectives and Contributions

- Clear project objectives
- Novel aspects (e.g., custom architecture, new activation function, optimizer comparison)
- Summary of technical contributions

4. Methodology

- Overall system pipeline
- Data preprocessing steps
- Training and testing split
- Evaluation strategy

5. Neural Network Model Descriptions

- DNN/DCNN architecture design for regression and classification
- Layer-wise model configuration
- Activation functions (standard vs. proposed)
- Optimization techniques (SGD + momentum, Adam)
- Loss functions and hyperparameters

6. Experiments and Results

6.1 Dataset Description

- Dataset source and structure
- Number of classes and samples
- Input dimensions

6.2 Training and Testing Performance

- Training and validation loss curves
- Accuracy and convergence behavior

6.3 Quantitative Evaluation Metrics

- Accuracy, Precision, Recall, F1-score
- ROC and Precision–Recall curves

6.4 Comparative Analysis

- Performance comparison across:
 - Activation functions
 - Optimizers
- Discussion of observed trends

6.5 Limitations and Challenges

- Computational constraints
- Dataset limitations
- Model generalization issues

7. Conclusion and Future Work

- Summary of findings
- Key takeaways
- Suggestions for future improvements or extensions

8. References

- Properly formatted academic references (APA / IEEE)

9. Appendix (Optional)

- Network hyperparameters
- Additional plots
- Pseudocode or algorithm descriptions